

E-Commerce API

Project Documentation

1. Category

Base path: /api/category/v1

Endpoints

Method	Endpoint	Description
GET	/getAllCats	Returns all categories
POST	/addCategory	Adds a new category
PUT	/updateCat/{id}	Updates an existing category
DELETE	/deleteCat/{id}	Deletes a category

Model Field Validations (Category)

- id: must not be empty ("id should not be empty")
- name: must not be empty ("name should not be empty"); must be at least 4 characters ("category name should be more than 3 letters")

Note: when a @Valid field validation fails, the API returns the raw message text as the response body (not wrapped in a message key). Only the first invalid field encountered is returned, even if multiple fields are invalid.

Endpoint Details

POST /addCategory — Returns 400 Bad Request when:

- id is empty
- name is empty
- name is shorter than 4 characters
- The id is already used by an existing category → "Category ID is already in use."
- A category with the same name already exists → "Category already exists."

PUT /updateCat/{id} — Returns 400 Bad Request when:

- id (body) is empty
- name is empty
- name is shorter than 4 characters
- The new id is already used by another category → "Category ID is already in use."
- A category with the same name already exists → "Category already exists."
- No category exists with the {id} in the path → "Category not found."

Known behavior: the service layer applies the update before the field-validation check is evaluated. If the path {id} exists but the request body fails @Valid validation, the record may already be modified internally even though the client receives a 400 response.

DELETE /deleteCat/{id} — Returns 400 Bad Request when:

- No category exists with the given {id} → "Category not found."

GET /getAllCats

Always returns 200 OK with the list of categories (no validation).

2. Merchant

Base path: /api/merchant/v1

Endpoints

Method	Endpoint	Description
GET	/getMerchants	Returns all merchants
POST	/addMerchant	Adds a new merchant
PUT	/updateMerchant/{id}	Updates an existing merchant
DELETE	/deleteMerchant/{id}	Deletes a merchant

Model Field Validations (Merchant)

- id: must not be empty ("id should not be empty")
- name: must not be empty ("name should not be empty"); must be at least 4 characters ("name should be more than 3 letters")

Endpoint Details

POST /addMerchant — Returns 400 Bad Request when:

- id is empty
- name is empty
- name is shorter than 4 characters
- The id is already used by an existing merchant → "Merchant ID is already in use."

PUT /updateMerchant/{id} — Returns 400 Bad Request when:

- id (body) is empty
- name is empty
- name is shorter than 4 characters
- The new id is already used by another merchant → "Merchant ID is already in use."
- No merchant exists with the {id} in the path → "Merchant not found."

Known behavior: same update-before-validation-check order as Category's update endpoint.

DELETE /deleteMerchant/{id} — Returns 400 Bad Request when:

- No merchant exists with the given {id} → "Merchant not found."

GET /getMerchants

Always returns 200 OK with the list of merchants.

3. Product

Base path: /api/product/v1

Endpoints

Method	Endpoint	Description
GET	/getProducts	Returns all products
POST	/addProducts	Adds a new product
PUT	/updateProduct/{id}	Updates an existing product
DELETE	/deleteProduct/{id}	Deletes a product
GET	/productsInRange/{categoryID}/{min}/{max}	Returns products in a category within a price range (Additional Endpoint)

Model Field Validations (Product)

- id: must not be empty ("id should not be empty")
- name: must not be empty ("name should not be empty"); must be at least 4 characters (no custom message set — the default Jakarta Bean Validation message is returned, e.g. "size must be between 4 and 2147483647")
- price: must not be null ("price should not be empty"); must be 0 or greater ("product price should be 0 or more")
- categoryID: must not be empty ("You should add a Category ID")

Endpoint Details

POST /addProducts — Returns 400 Bad Request when:

- id is empty
- name is empty
- name is shorter than 4 characters
- price is null
- price is negative
- categoryID is empty
- The id is already used by an existing product → "Product ID is already in use."
- The categoryID does not match any existing category → "Category not found."

PUT /updateProduct/{id} — Returns 400 Bad Request when:

- id (body) is empty
- name is empty
- name is shorter than 4 characters

- price is null
- price is negative
- categoryID is empty
- The new id is already used by another product → "Product ID is already in use."
- The categoryID does not match any existing category → "Category not found."
- No product exists with the {id} in the path → "Product not found."

Note: unlike the other classes' update endpoints, updateProduct checks @Valid field errors before calling the service layer, so an invalid request never touches the stored data.

DELETE /deleteProduct/{id} — Returns 400 Bad Request when:

- No product exists with the given {id} → "Product not found."

GET /productsInRange/{categoryID}/{min}/{max} — Returns 400 Bad Request when:

- min is negative, or min is greater than max → "Minimum price must be 0 or higher and less than or equal to the maximum price."
- No products exist in that category within the given price range → "No products found in this category within the specified price range."

GET /getProducts

Always returns 200 OK with the list of products.

4. MerchantStock

Base path: /api/merchantStock/v1

Endpoints

Method	Endpoint	Description
GET	/getStocks	Returns all merchant stock records
POST	/addStock	Adds a new stock record
PUT	/updateEntireStock/{id}	Fully updates a stock record
DELETE	/deleteStock/{id}	Deletes a stock record
PUT	/updateStock/{productID}/ {merchantID}/{stock}	Updates only the stock quantity (Additional Endpoint)
PUT	/applyDiscount/{merchantID}/ {productID}/{discountPercentage}	Applies a discount to a merchant's product (Additional Endpoint)

Model Field Validations (MerchantStock)

- id: must not be empty ("id should not be empty")
- productId: must not be empty ("product id should not be empty")
- merchantID: must not be empty ("merchant id should not be empty")
- stock: must not be null ("stock cannot be empty"); must be at least 10 ("stock should starts from 10")

Endpoint Details

POST /addStock — Returns 400 Bad Request when:

- id is empty
- productId is empty
- merchantID is empty
- stock is null or less than 10
- The id is already used by an existing stock record → "Stock ID is already in use."
- The productId does not match any existing product → "Product not found."
- The merchantID does not match any existing merchant → "Merchant not found."
- This merchant already has a stock record for this product → "This merchant already has this product in stock."

Note: field validation (e.g. $stock \geq 10$) is checked before any business logic, so a request with an invalid stock value returns the field-validation message even if the product/merchant IDs are also invalid.

PUT /updateEntireStock/{id} — Returns 400 Bad Request when:

- id (body) is empty
- productId is empty

- merchantID is empty
- stock is null or less than 10
- The new id is already used by another stock record → "Stock ID is already in use."
- The productID does not match any existing product → "Product not found."
- The merchantID does not match any existing merchant → "Merchant not found."
- This merchant already has a stock record for this product → "This merchant already has this product in stock."
- No stock record exists with the {id} in the path → "Stock not found."

Known behavior: same update-before-validation-check order as Category/Merchant.

DELETE /deleteStock/{id} — Returns 400 Bad Request when:

- No stock record exists with the given {id} → "Stock not found."

PUT /updateStock/{productID}/{merchantID}/{stock} — Returns 400 Bad Request when:

- The productID does not match any existing product → "Product not found."
- The merchantID does not match any existing merchant → "Merchant not found."
- The resulting stock quantity would be 10 or less → "Stock quantity must be greater than 10."

PUT /applyDiscount/{merchantID}/{productID}/{discountPercentage} — Returns 400 Bad Request when:

- The productID does not match any existing product → "Invalid product ID."
- The merchantID does not match any existing merchant → "Invalid merchant ID."
- discountPercentage is less than 5 → "Discount percentage must be 5% or higher."
- This merchant does not sell this product → "This merchant does not sell this product."

GET /getStocks

Always returns 200 OK with the list of stock records.

5. User

Base path: /api/user/v1

Endpoints

Method	Endpoint	Description	Access
GET	/getUsers	Returns all users	Open (no caller check)
POST	/addUser	Registers a new user	Open (no caller check)
PUT	/updateUser/{id}	Updates a user	Open (no caller check)
DELETE	/deleteUser/{id}	Deletes a user	Open (no caller check)
PUT	/buyProduct/{userID}/ {productID}/{merchantID}	Purchases a product (Additional Endpoint)	Customers only — admins are blocked
PUT	/giveBalance/{senderID}/ {recipientID}/{amount}	Sends gift-card balance to another user (Additional Endpoint)	Any existing user may send; recipient cannot be an admin
PUT	/returnProduct/{userID}/ {productID}	Returns a previously purchased product (Additional Endpoint)	Customers only — admins are blocked
GET	/PreviousOrders/{userID}	Lists a user's previously purchased products (Additional Endpoint)	Any existing user (own orders only, via own ID)
GET	/displayCustomer/{userID}	Lists all customer accounts (Extra Credit Endpoint)	Admins only

Model Field Validations (User)

- id: must not be empty ("id should not be empty")
- username: must not be empty ("username should not be empty"); at least 6 characters ("username should be more than 5 characters")
- password: must not be empty ("password should not be empty"); must be at least 6 characters and include at least one uppercase letter, one lowercase letter, one digit, and one special character from @\$%^&+=! ("Password must be 6+ characters and include one uppercase letter, one lowercase letter, one number, and one special character.")
- email: must not be empty ("email should not be empty"); must be a valid email format ("email format is invalid")
- role: must not be empty ("role is required"); must be exactly "customer" or "admin" ("Role must be either 'customer' or 'admin'")
- balance: must not be null ("balance is required"); must be 0 or greater ("balance should be 0 or more")

Endpoint Details

POST /addUser — Returns 400 Bad Request when:

- id is empty
- username is empty or shorter than 6 characters
- password is empty, shorter than 6 characters, or missing an uppercase letter, a lowercase letter, a digit, or a special character (@#\$\$%^&+=!)
- email is empty or not a valid email format
- role is empty or is not exactly "customer" or "admin"
- balance is null or negative
- The id is already used by an existing user → "User ID is already in use."
- The email is already used by an existing account → "An account with this email already exists."
- role is "admin" and balance is not exactly 0 → "Admin balance must be zero."

PUT /updateUser/{id} — Returns 400 Bad Request when:

- Same field validations as addUser above
- The new id is already used by another user → "User ID is already in use."
- The new email is already used by another user → "This email is already in use."
- No user exists with the {id} in the path → "User not found."
- role is "admin" and balance is not exactly 0 → "Admin balance must be zero."

Known behavior: same update-before-validation-check order as Category/Merchant/MerchantStock.

DELETE /deleteUser/{id} — Returns 400 Bad Request when:

- No user exists with the given {id} → "User not found."

PUT /buyProduct/{userID}/{productID}/{merchantID} — Returns 400 Bad Request when:

- userID does not match any existing user → "User not found."
- productID does not match any existing product → "Product not found."
- merchantID does not match any existing merchant → "Merchant not found."
- This merchant does not sell this product → "This merchant does not sell this product."
- The product is out of stock (0 units) with this merchant → "This product is currently out of stock."
- The user's balance is lower than the product's price → "Insufficient balance to purchase this product."
- The user with userID has the admin role → "Administrators cannot purchase products."

PUT /giveBalance/{senderID}/{recipientID}/{amount} — Returns 400 Bad Request when:

- senderID does not match any existing user → "Sender not found."
- recipientID does not match any existing user → "Recipient not found."
- The sender's balance is lower than amount → "Insufficient balance for this transaction."
- The recipient has the admin role → "The recipient cannot be an administrator."

PUT /returnProduct/{userID}/{productID} — Returns 400 Bad Request when:

- userID does not match any existing user → "User not found."

- productID does not match any existing product → "Product not found."
- The user with userID has the admin role → "Administrators cannot return products."
- The user has not purchased this product (or already returned it) → "You have not purchased this product."

GET /PreviousOrders/{userID} — Returns 400 Bad Request when:

- userID does not match any existing user → "User not found."
- The user has no products in their purchase history → "No previous orders found."

GET /displayCustomer/{userID} — Returns 400 Bad Request when:

- userID does not match any existing user, or the user exists but does not have the admin role → "Only administrators can access this information."
- On success, returns the list of all accounts with the customer role.

GET /getUsers

Always returns 200 OK with the list of users.

Postman Testing Guide

Only test cases confirmed with an actual Postman screenshot during testing are included below, organized by class. Test data reflects a realistic Amazon-style dataset built up over the course of testing.

Category

#	Method & URL	Body	Result
1	POST /api/category/v1/addCategory	{"id":"cat1","name":"Electronics"}	200 — "Category Electronics has been added successfully"
2	POST /api/category/v1/addCategory	{"id":"cat2","name":"TV"}	400 — "category name should be more than 3 letters"
3	POST /api/category/v1/addCategory	{"id":"cat1","name":"Home Appliances"}	400 — "Category ID is already in use."
4	GET /api/category/v1/getAllCats	—	200 — list containing cat1
5	PUT /api/category/v1/updateCat/cat1	{"id":"cat1","name":"Electronics & Gadgets"}	200 — "Category has been updated successfully."
6	PUT /api/category/v1/updateCat/cat99	{"id":"cat99","name":"Ghost Category"}	400 — "Category not found."
7	DELETE /api/category/v1/deleteCat/cat1	—	200 — "Category has been deleted successfully."

Merchant

#	Method & URL	Body	Result
8	POST /api/merchant/v1/addMerchant	{"id":"merch1","name":"TechStore"}	200 — "Merchant TechStore has been added successfully."
9	POST /api/merchant/v1/addMerchant	{"id":"merch2","name":"JBL"}	400 — "name should be more than 3 letters"
10	POST /api/merchant/v1/addMerchant	{"id":"merch1","name":"TechStore Outlet"}	400 — "Merchant ID is already in use."
11	GET /api/merchant/v1/getMerchants	—	200 — list containing merch1
12	PUT /api/merchant/v1/updateMerchant/merch1	{"id":"merch1","name":"TechStore Plus"}	200 — "Merchant has been updated successfully."
13	PUT /api/merchant/v1/updateMerchant/merch99	{"id":"merch99","name":"Ghost Store"}	400 — "Merchant not found."

#	Method & URL	Body	Result
14	DELETE /api/merchant/v1/deleteMerchant/merch1	—	200 — "Merchant has been deleted successfully."
15	DELETE /api/merchant/v1/deleteMerchant/merch1	—	400 — "Merchant not found."

Product

#	Method & URL	Body	Result
16	POST /api/product/v1/addProducts	{"id":"prod1","name":"MacBook Air M2","price":4299,"categoryID":"cat1"}	200 — "Product has been added successfully."
17	POST /api/product/v1/addProducts	{"id":"prod2","name":"Logitech MX Master 3S","price":349,"categoryID":"catX"}	400 — "Category not found."
18	POST /api/product/v1/addProducts	{"id":"prod3","name":"Refurbished iPad","price":-50,"categoryID":"cat1"}	400 — "product price should be 0 or more"
19	POST /api/product/v1/addProducts	{"id":"prod1","name":"MacBook Air M3","price":4599,"categoryID":"cat1"}	400 — "Product ID is already in use."
20	GET /api/product/v1/getProducts	—	200 — list containing prod1
21	PUT /api/product/v1/updateProduct/prod1	{"id":"prod1","name":"MacBook Air M2 - 256GB","price":3999,"categoryID":"cat1"}	200 — "Product has been updated successfully."
22	PUT /api/product/v1/updateProduct/prod99	{"id":"prod99","name":"Ghost Product","price":100,"categoryID":"cat1"}	400 — "Product not found."
23	GET /api/product/v1/productsInRange/cat1/1000/5000	—	200 — list containing prod1
24	GET /api/product/v1/productsInRange/cat1/5000/1000	—	400 — "Minimum price must be 0 or higher and less than or equal to the maximum price."
25	DELETE /api/product/v1/deleteProduct/prod9	—	200 — "Product has been deleted successfully."
26	DELETE /api/product/v1/deleteProduct/prod9	—	400 — "Product not found."

MerchantStock

#	Method & URL	Body	Result
27	POST /api/merchantStock/v1/addStock	{"id":"stock1","productId":"prod1","merchantID":	200 — "Merchant

#	Method & URL	Body	Result
		"merch1","stock":25}	stock has been added successfully."
28	POST /api/merchantStock/v1/addStock	{"id":"stock2","productId":"prodX","merchantID":"merch1","stock":15}	400 — "Product not found."
29	POST /api/merchantStock/v1/addStock	{"id":"stock3","productId":"prod1","merchantID":"merchX","stock":15}	400 — "Merchant not found."
30	POST /api/merchantStock/v1/addStock	{"id":"stock3","productId":"prod1","merchantID":"merchX","stock":9}	400 — "stock should starts from 10" (field validation runs before the business-logic check)
31	GET /api/merchantStock/v1/getStocks	—	200 — list containing stock1
32	POST /api/merchantStock/v1/addStock	{"id":"stock4","productId":"prod1","merchantID":"merch1","stock":20}	400 — "This merchant already has this product in stock."
33	PUT /api/merchantStock/v1/updateEntireStock/stock1	{"id":"stock1","productId":"prod1","merchantID":"merch1","stock":40}	200 — "Stock has been updated successfully."
34	PUT /api/merchantStock/v1/updateEntireStock/stock99	{"id":"stock1","productId":"prod1","merchantID":"merch1","stock":40}	400 — "Stock not found."
35	PUT /api/merchantStock/v1/updateStock/prod1/merch1/50	—	200 — "Stock has been updated successfully."
36	PUT /api/merchantStock/v1/applyDiscount/merch1/prod1/10	—	200 — "Discount has been applied successfully."

User

Note: the password fields shown below (e.g. "Admin2026", "Nourah2026") were valid at the time these tests were run. The password validation rule was strengthened afterward (now requiring a special character), so these historical passwords no longer satisfy the current rule described in the User section above.

#	Method & URL	Body	Result
37	POST /api/user/v1/addUser	{"id":"admin1","username":"adminFahad","password":"Admin2026","email":"fahad.admin@amazon-clone.com","role":"admin","balance":0}	200 — "User adminFahad has been added successfully."
38	POST /api/user/v1/addUser	{"id":"admin2","username":"adminSara","password":"Admin2027","email":"sara.admin@amazon-clone.com","role":"admin","balance":100}	400 — "Admin balance must be zero."
39	POST /api/user/v1/addUser	{"id":"user1","username":"NourahBuyer","password":"Nourah2026","email":"Nourah.buyer@gmail.com","role":"customer","balance":5000}	200 — "User NourahBuyer has been added"

#	Method & URL	Body	Result
			successfully."
40	PUT /api/user/v1/buyProduct/user1/prod1/ merch1	—	200 — "Your order has been completed successfully."
41	PUT /api/user/v1/buyProduct/user2/prod1/ merch1	—	400 — "Insufficient balance to purchase this product."
42	PUT /api/user/v1/buyProduct/admin1/prod1/ merch1	—	400 — "Administrators cannot purchase products."
43	PUT /api/user/v1/giveBalance/user1/user2/1 00	—	200 — "Gift card balance has been sent successfully to the user with ID: user2"
44	PUT /api/user/v1/returnProduct/user1/prod 1	—	200 — "Product has been returned successfully."
45	PUT /api/user/v1/returnProduct/user2/prod 1	—	400 — "You have not purchased this product."
46	GET /api/user/v1/PreviousOrders/user2	—	400 — "No previous orders found."
47	GET /api/user/v1/displayCustomer/admin1	—	200 — list of customer accounts (user1, user2)
48	GET /api/user/v1/displayCustomer/user2	—	400 — "Only administrators can access this information."